Reviewer Pack — Minimal Rank of Cyclic p-Adic Representations and Communicat…

Entry 1d74bff40233 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 21:57:43 UTC

Minimal Rank of Cyclic p-Adic Representations and Communication Complexity Rank Correlation

Entry ID: `1d74bff40233` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-02 21:57:43 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Communication Complexity

Statement:

> For any given communication complexity problem with n bits, the minimal rank of the cyclic p-adic representation of its underlying linear code is linearly correlated with its communication complexity rank, such that R(n) = Θ(r(n)), where r(n) is the communication complexity rank and R(n) is the minimal rank of the cyclic p-adic representation.

Rationale (proposer's reasoning):

> The bridge between p-adic analysis and communication complexity could expose a new perspective on the structure of linear codes underlying communication protocols, potentially leading to better understanding of lower bounds for communication complexity. The use of cyclic p-adic representations provides a novel mathematical tool to analyze these codes, which has not been extensively explored in the context of complexity theory.

Taxonomy category: `p-adic_analysis_communication_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a5135399c6e7972b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40, the correlation coefficient between the minimal rank R(n) and communication complexity rank r(n) across 30 random seeds is at least 0.7, with no seed having a correlation below 0.5.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    augmented_matrix = [row[:] + [1] for row in matrix]
    rank = 0

    for i in range(n):
        if i >= len(augmented_matrix): break

        # Find pivot
        pivot_row = i
        while pivot_row < n and augmented_matrix[pivot_row][i] == 0:
            pivot_row += 1
        if pivot_row == n: continue

        # Swap rows
        augmented_matrix[i], augmented_matrix[pivot_row] = augmented_matrix[pivot_row], augmented_

        # Eliminate below the pivot
        for j in range(i + 1, n):
            factor = Fraction(augmented_matrix[j][i], augmented_matrix[i][i])
            for k in range(n + 1):
                augmented_matrix[j][k] -= factor * augmented_matrix[i][k]

    # Count non-zero rows
    rank = sum(1 for row in augmented_matrix if any(row[k] != 0 for k in range(n)))

    return matrix, augmented_matrix, rank

def minimal_p_adic_rank(code):
    n = len(code)
    p = 2  # Using binary representation for simplicity

    # Convert code to a matrix
    A = [[int(bit) for bit in codeword] for codeword in code]
```

```
    _, _, rank = gaussian_elimination(A)
    return rank

def communication_complexity_rank(code):
    n = len(code)
    p = 2  # Using binary representation for simplicity

    # Convert code to a matrix
    A = [[int(bit) for bit in codeword] for codeword in code]

    _, _, rank = gaussian_elimination(A)
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    n_max = 40

    min_ranks = []
    comm_complexity_ranks = []

    for _ in range(instances_tested):
        code_length = random.randint(5, 10)
        code = [''.join(random.choice('01') for _ in range(code_length)) for _ in range(n)]

        min_rank = minimal_p_adic_rank(code)
        comm_complexity_rank_value = communication_complexity_rank(code)

        min_ranks.append(min_rank)
        comm_complexity_ranks.append(comm_complexity_rank_value)

    correlation_coefficient = sum((min_ranks[i] - sum(min_ranks) / instances_tested) * (comm_compl

    conjecture_holds = correlation_coefficient >= 0.7
    counterexample = "" if conjecture_holds else "correlation_coefficient < 0.7"

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys

    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
```

```
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = (sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results)) *
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.7\" first_failing_s
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3ef9f9a.py", line 106, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3ef9f9a.py", line 79, in run_trial
    min_rank = minimal_p_adic_rank(code)
               ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3ef9f9a.py", line 53, in minimal_p_adi
    _, _, rank = gaussian_elimination(A)
                 ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c3ef9f9a.py", line 39, in gaussian_elim
    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    ~~~~~~~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	20520
2	preregistration	ollama_remote	glm4:latest	0	0	16748
3	novelty	ollama_remote	glm4:latest	0	0	12124
4	novelty	ollama_remote	glm4:latest	0	0	8809
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19224
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15606
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25099
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12365
9	judge	ollama_remote	glm4:latest	0	0	11937

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 142432 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1d74bff40233.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1d74bff40233.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1d74bff40233.tar.gz` (if generated)